claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-aa58f4ccaf67eca9d.jsonl`
- SHA-256: `5a100c2a412106c52bb49f875d2127473cec9b98bb3ca6ab5dde986192ad3520`
- Source modified: `2026-07-04T09:05:56+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_8c732492-138`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`

Transcript

1. user (2026-07-04T08:53:12.154Z)

You are an ADVERSARIAL verifier for a fix to PR #467 (branch fix/upload-gcs-staging) of Khelsutra/badminton-highlight-indexer. A fix agent just left UNCOMMITTED changes in the worktree C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging. Your default stance: try to REFUTE that the fix is correct and complete.

The blocker it must resolve:
REVIEWER BLOCKER (verbatim, from PR #467):
"I found one more config-shape blocker in the staging gate.
The new local-upload staging path decides whether staging is possible with:
    input_root = storage_cfg.input_root
    input_backend = storage_cfg.input_backend or 'local'
    if not input_root or input_backend == 'local':
        return _failed(...)
That rejects a valid config where storage.input_root is itself an explicit cloud URI and input_backend is left at its default. The storage layer already treats an explicit root like gs://in-bucket/videos as backend-bearing (resolve_input_uri joins under input_root before consulting input_backend), and the model docs call out input_root='gs://bucket/videos' + 'x.mp4' => 'gs://bucket/videos/x.mp4'.
I reproduced on this head with:
    {"deployment": {"compute_target": "cloud_run"}, "storage": {"backend": "gcs", "input_root": "gs://in-bucket/videos", "output_root": "gs://out-bucket/results"}}
A local upload still fails pre-dispatch with 'needs a cloud input ... or a cloud storage.input_root' because input_backend remains the default local, even though input_root is already a gs:// destination.
Suggested fix: classify the configured input_root itself (e.g. storage.parse_uri(input_root).backend in {'gcs', 's3', 'gdrive-api'} / at least not 'local') instead of relying on input_backend, and add a regression test where input_root='gs://...' is set without input_backend='gcs'."

The fix agent reported:
- files changed: ["C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging/backend/orchestration.py","C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging/tests/test_api_cloud_dispatch.py"]
- tests added: ["tests/test_api_cloud_dispatch.py :: test_cloud_stages_when_input_root_is_cloud_and_input_backend_default"]
- suite: 1629 passed, 10 skipped, 8 warnings in 128.78s (0:02:08) | ruff: ruff 0.15.16 ? "All checks passed!" (python -m ruff check ., from worktree root) | mypy: mypy 2.1.0 ? "Success: no issues found in 147 source files" (python -m mypy backend training; 4 pre-existing annotation-unchecked notes only)
- summary: Replaced the staging gate's predicate in backend/orchestration.py (~line 254-267, inside the input_ref.backend == 'local' branch): instead of "not input_root or input_backend == 'local'", the gate now classifies the configured input_root itself via storage.parse_uri(input_root).backend, failing only when that is 'local' (empty root or an unsupported-scheme root are treated as local, the latter via a caught ValueError). This is the

truthful gate because staged_uri is built by joining under input_root and storage.put() routes by parse_uri(staged_uri).backend ? so an explicit cloud root (gs://, s3://) stages fine with input_backend at its default, exactly the reviewer's repro. The legacy shape the old gate accepted (bare/local-looking input_root + explicit non-local input_backend) was broken downstream anyway: put() would route to LocalStorage (a copy the Cloud Run job can't read) and the follow-up resolve_input_ref(staged_uri) would double-join the bare staged path under input_root, so classifying the root removes a lie rather than a capability. Added the required regression test mimicking the existing staging test's fixture pattern (input_root='gs://in-bucket/videos', input_backend asserted still at default 'local', local upload -> storage.put called with the gs:// staged URI and the Cloud Run dispatch continues from it). The existing negative test (no input_root -> 'needs a cloud input' failure, no put, no dispatch) still passes unchanged. Changes are left uncommitted in the worktree as instructed.
- problems:

DO (read-only on the worktree except where stated):
1. Read the full diff: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging diff (plus git -C C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging status --short for new files). Check the fix logic line by line against the blocker. Hunt for: edge cases the reviewer's repro implies but the fix misses, behavior changes beyond the blocker's scope, style that clashes with the repo, docstrings that overstate.
2. PROVE the regression test bites: create a THROWAWAY detached worktree at the PR head: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging worktree add C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-467 d3a9629c9e1894304a4f97ccd631dd5221fc54fc --detach ; copy ONLY the new/changed test file(s) from C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging into it; run those specific tests there with python -m pytest <testfile> -q -p no:cacheprovider ? they MUST FAIL (that proves the test catches the original bug). Then remove the throwaway: git -C C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging worktree remove --force C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd/scratchpad/verify-467. If the test PASSES at the unfixed head, that is a refutation ? report it.
3. Run the new tests in the REAL worktree to confirm they pass with the fix: python -m pytest <testfile> -q -p no:cacheprovider.
4. Do NOT modify the fix agent's changes in C:/Users/avidu/Projects/khelsutra-guru/wt-upload-staging. Do NOT commit or push anywhere. PowerShell tool on Windows; global python 3.13.
5. For PR #468 specifically (if that is your PR): also re-run the new concurrency test 5 times in a row in the real worktree to smoke out flakiness, and report.

Fill the schema honestly. verdict='approve' ONLY if the fix fully resolves the blocker, the test provably bites, and you found no real issues.

2. user (2026-07-04T08:53:20.257Z)

```
 M backend/orchestration.py
 M tests/test_api_cloud_dispatch.py
d3a9629 review: thread active storage config into the staging put (#461)
```

3. user (2026-07-04T08:53:21.442Z)

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 46be449..4f9d4ea 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -251,11 +251,20 @@ def _maybe_dispatch_remote(
         # A browser upload lands on THIS box's local disk; the Cloud Run job (a different machine)
         # can't read it. Stage it UP to the input bucket first, then dispatch from there (#461). Only
         # possible when a CLOUD input_root is configured; without one we genuinely can't stage ? fail.
+        # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged URI below is
+        # built by joining under input_root, and storage.put routes by parse_uri(staged_uri).backend ?
```

```
+        # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left at its
+        # default, while a bare/local-looking root can't stage no matter what input_backend
says
+        # (resolve_input_uri only consults input_backend when input_root is EMPTY, and an empty
root
+        # gives us nothing to build the staged URI under).
         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
-        input_backend = (
-            getattr(storage_cfg, "input_backend", "local") or "local"
-        ).strip()
-        if not input_root or input_backend == "local":
+        try:
+            root_backend = (
+                storage.parse_uri(input_root).backend if input_root else "local"
+            )
+        except ValueError:  # unsupported scheme in input_root ? nowhere real to stage to
+            root_backend = "local"
+        if root_backend == "local":
             return _failed(
                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud storage.input_root
to "
                 "stage local uploads into; a path local to this machine can't be read by the
Cloud "
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index ee36ca1..7b67de0 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -274,6 +274,43 @@ def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path,
monkeypatch):
     assert row is not None and row["filepath"] == dest


+def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
+    cloud_env, tmp_path, monkeypatch
+):
+    """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid cloud
config ?
+    the storage layer classifies an explicit root by its scheme (resolve_input_uri joins under
+    input_root before ever consulting input_backend), so the staging gate must too (PR #467
review).
+    Regression: the gate previously required input_backend != 'local' and hard-failed this
shape."""
+    _db, _bucket, _mp = cloud_env
+    m.global_config.storage.input_root = "gs://in-bucket/videos"
+    # input_backend deliberately NOT set ? stays at the model default 'local'
+    assert m.global_config.storage.input_backend == "local"
+    puts: list = []
+
+    def _fake_put(src, uri, **kw):
+        puts.append((str(src), uri))
+        return uri
+
+    monkeypatch.setattr(m.storage, "put", _fake_put)
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    local_path = str(tmp_path / "upload.mp4")
+
+    out = m._execute_processing(
+        m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
+    )
+
+    assert out["status"] == "success"  # NOT the 'needs a cloud input' pre-dispatch failure
+    assert len(puts) == 1
+    src, dest = puts[0]
+    assert src == local_path
```

```
+    assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
+    # dispatch continued from the STAGED gs:// URI
+    assert len(fake.calls) == 1
+    argv = fake.calls[0]
+    assert argv[argv.index("--video-uri") + 1] == dest
+
+
 def test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path, monkeypatch):
     """No cloud input_root configured ? we genuinely can't stage a local upload ? hard-fail (no
     dispatch), rather than dispatching a path the Cloud Run job can't read."""
```

4. user (2026-07-04T08:53:32.359Z)

```
200                     "status": "failed",
201                     "message": msg,
202                     "results": [r.model_dump() for r in val_results],
203                     "play_segments": [],
204                     "compute": {
205                         "path": "cloud_run" if ran else "not_run",
206                         "requested": req.compute,
207                         "target": "cloud_run",
208                         "dispatched": ran,
209                     },
210                 },
211                 ran,
212             )
213
214         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
215         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
216         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
217         choice = (req.compute or "").strip().lower()
218         if choice and choice not in ("local", "cloud"):
219             logger.warning(
220                 "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
221                 req.compute,
222             )
223             choice = ""
224         if choice == "local":
225             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
226         if choice == "cloud":
227             if not _cloud_available(cfg):
228                 return _failed(
229                     "cloud-serving was requested but this server isn't configured for it "
230                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
231                     False,
232                 )
233             compute = get_compute_backend(cfg, target_override="cloud_run")
234         else:
235             compute = get_compute_backend(
236                 cfg
237             )  # server default (default-OFF unless cloud_run)
238         if compute is None:
239             return None  # default-OFF ? run the in-process segmenter below (unchanged)
240
241         storage_cfg = getattr(cfg, "storage", None)
242
243         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
244         try:
```

```
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
            # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
            # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
            # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
            # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
            # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
            # default, while a bare/local-looking root can't stage no matter what
input_backend says
            # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
            # gives us nothing to build the staged URI under).
            input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
            try:
                root_backend = (
                    storage.parse_uri(input_root).backend if input_root else "local"
                )
            except ValueError:  # unsupported scheme in input_root ? nowhere real to stage
to
                root_backend = "local"
            if root_backend == "local":
                return _failed(
                    "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
                    "stage local uploads into; a path local to this machine can't be read by
the Cloud "
                    "Run job.",
                    False,
                )
            local_src = input_ref.key  # resolved local path of the upload
            staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
            try:
                notify("staging upload to bucket")
                # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
                # region/addressing for R2·DO·MinIO; GCS project/client) ? not a fresh
default client.
                storage.put(
                    local_src,
                    staged_uri,
                    config=(
                        storage_cfg.model_dump() if storage_cfg is not None else None
                    ),
                )
            except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
                logger.error(
                    "[API] cloud-serving: failed to stage %s -> %s: %s",
                    local_src,
                    staged_uri,
                    e,
                    exc_info=True,
                )
```

```
295                 return _failed(
296                     f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
297                     False,
298                 )
299             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
300             # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
301             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
302             req.video_path = staged_uri
303             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
304             db.add_video(
305                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
306             )
307         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
308         if not out_root:
309             return _failed(
310                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
311                 "? that's where the Cloud Run job writes the result.",
312                 False,
313             )
314
315         notify("dispatching (cloud_run)")
316         logger.info(
317             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
318             req.video_path,
319             out_root,
```

5. user (2026-07-04T08:53:33.656Z)

```
<tool_use_error>Path does not exist: C:/Users/avidu/Projects/khelsutra-guru/wt-upload-
staging/backend/storage.py. Note: your current working directory is
C:\Users\avidu\Projects\khelsutra-guru\wt-dockerfile-auth.</tool_use_error>
```

6. user (2026-07-04T08:53:39.420Z)

```
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\__init__.py:50:def
resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\__init__.py:182:def
put(
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\s3.py:57:    def
put(self, local_path, ref, *, content_type=None) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:51:def
parse_uri(uri: str) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:75:def
resolve_input_uri(
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\gcs.py:50:    def
put(self, local_path, ref, *, content_type=None) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\gdrive_api.py:181:
def put(self, local_path, ref, *, content_type=None) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\gdrive.py:131:    def
put(self, local_path, ref, *, content_type=None) -> StorageRef:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\base.py:34:    def put(
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\local.py:43:    def
put(self, local_path, ref, *, content_type=None) -> StorageRef:
```

7. user (2026-07-04T08:53:44.726Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
```

```
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14
15      from __future__ import annotations
16
17      import re
18      from dataclasses import dataclass
19      from datetime import datetime
20      from typing import Optional
21
22      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
23      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
32          etag: Optional[str] = None
33          content_type: Optional[str] = None
34
35
36      @dataclass(frozen=True)
37      class StorageRef:
38          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
39          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
40
41          backend: str
42          bucket: str
43          key: str
44          uri: str
45
46          @property
47          def name(self) -> str:
48              return self.key.rstrip("/").rsplit("/", 1)[-1]
49
50
51      def parse_uri(uri: str) -> StorageRef:
52          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
53
54          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
55          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
56          """
57          s = str(uri)
58          m = _SCHEME_RE.match(s)
59          if not m:
60              return StorageRef("local", "", s, f"local://{s}")
61          scheme = m.group(1).lower()
```

```
62          rest = s[m.end() :]
63          if scheme == "gs":
64              scheme = "gcs"
65          if scheme in ("local", "gdrive", "gdrive-api"):
66              # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
67              # root folder (gdrive-api resolves the path ? a Drive file id).
68              return StorageRef(scheme, "", rest, s)
69          if scheme in _KNOWN:
70              bucket, _, key = rest.partition("/")
71              return StorageRef(scheme, bucket, key, s)
72          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
73
74
75      def resolve_input_uri(
76          raw: str, *, input_backend: str = "local", input_root: str = ""
77      ) -> str:
78          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
79
80          Precedence (first match wins):
81            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
82            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
83            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
84               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
85            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
86                ``"gcs://bucket/x"``).
87            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
88
89          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
90          path used today, preserving byte-for-byte behaviour.
91          """
92          s = str(raw)
93          if _SCHEME_RE.match(s):  # 1. explicit scheme wins
94              return s
95          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
96          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
97          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
98          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
99              return s
100         if input_root:  # 3. join a relative name under the configured root
101             return input_root.rstrip("/") + "/" + s.lstrip("/")
102         if (
103             input_backend and input_backend != "local"
104         ):  # 4. prefix a bucket/key with the declared backend
105             return f"{input_backend}://{s}"
106         return s  # 5. bare/relative local path ? unchanged
107
```

8. user (2026-07-04T08:53:45.220Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
```

```
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13
14      from __future__ import annotations
15
16      import os
17      import shutil
18      import tempfile
19      from typing import Any, Dict, Iterator, Optional
20
21      from backend.storage.base import StorageBackend
22      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24      __all__ = [
25          "StorageBackend",
26          "StorageRef",
27          "StorageStat",
28          "parse_uri",
29          "resolve_input_uri",
30          "get_storage",
31          "fetch",
32          "put",
33          "list_uris",
34          "acquire_local_input",
35          "cleanup_acquired_local_input",
36          "input_is_local",
37          "resolve_input_ref",
38      ]
39
40
41      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
42          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
43          if storage_config is None:
44              return {}
45          if hasattr(storage_config, "model_dump"):
46              return storage_config.model_dump()  # type: ignore[no-any-return]
47          return dict(storage_config)
48
49
50      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
51          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
52          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
53          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
54
55          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
56          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
57          sc = _storage_dict(storage_config)
58          uri = resolve_input_uri(
59              raw,
60              input_backend=sc.get("input_backend", "local"),
61              input_root=sc.get("input_root", ""),
62          )
63          return parse_uri(uri)
```

```
64
65
66      def input_is_local(raw: str, storage_config: Any = None) -> bool:
67          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
68          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
69          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
70          ``resolve_input_ref``)."""
71          return resolve_input_ref(raw, storage_config).backend == "local"
72
73
74      def acquire_local_input(
75          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
76      ) -> str:
77          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
78
79          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
80          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
81          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
82          path returned. The caller owns successful remote scratch copies; use
83          ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
84          sc = _storage_dict(storage_config)
85          ref = resolve_input_ref(raw, sc)
86          if ref.backend == "local":
87              return (
88                  ref.key
89              )  # identity ? no temp, no copy, byte-identical to passing the path directly
90          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
91          try:
92              return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
93          except Exception:
94              shutil.rmtree(scratch, ignore_errors=True)
95              raise
96
97
98      def cleanup_acquired_local_input(
99          raw: str, local_path: str, storage_config: Any = None
100     ) -> None:
101         """Delete a remote scratch dir produced by ``acquire_local_input``.
102
103         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
104         into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
105         finished hashing/validating/stitching/segmenting the returned file.
106         """
107         try:
108             if input_is_local(raw, storage_config):
109                 return
110         except ValueError:
111             return
112         if not local_path:
113             return
114         scratch = os.path.dirname(os.path.abspath(local_path))
115         if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
116             shutil.rmtree(scratch, ignore_errors=True)
117
118
119     def _backend_class(name: str):
```

```
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage

            return GDriveStorage
        if name == "gdrive-api":
            from backend.storage.gdrive_api import GDriveApiStorage

            return GDriveApiStorage
        raise ValueError(
            f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
        )


    def get_storage(
        backend: Optional[str] = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        **overrides,
    ) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
        # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
        # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
        settings: Dict[str, Any] = dict(
            cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
        )
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(
        uri: str,
        dst: "str | os.PathLike[str] | None" = None,
        *,
        config: Optional[Dict[str, Any]] = None,
        overwrite: bool = False,
    ) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(
            ref, dst, overwrite=overwrite
```

```
179         )
180
181
182     def put(
183         local_path: "str | os.PathLike[str]",
184         uri: str,
185         *,
186         config: Optional[Dict[str, Any]] = None,
187         content_type: Optional[str] = None,
188     ) -> str:
189         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
190         ref = parse_uri(uri)
191         get_storage(ref.backend, config=config).put(
192             local_path, ref, content_type=content_type
193         )
194         return uri
195
196
197     def list_uris(
198         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
199     ) -> Iterator[str]:
200         """Yield the URI of every object under ``prefix_uri``."""
201         ref = parse_uri(prefix_uri)
202         for r in get_storage(ref.backend, config=config).list(ref):
203             yield r.uri
204
```

9. assistant (2026-07-04T08:54:14.218Z)

Now let me look at the test file to check the fixture pattern and whether the config mutation leaks.

10. user (2026-07-04T08:54:14.493Z)

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
9
10      import csv
11      import json
12      import os
13
14      import pytest
15
16      pytest.importorskip("httpx")
17      from fastapi.testclient import TestClient
18
19      import backend.compute as compute_pkg
20      import backend.main as m
21      from backend.compute.cloud_run import CloudRunJobClient
22      from backend.config.models import AppConfig
23      from backend.infrastructure.database import Database
24      from backend.infrastructure.execution_policy import ExecutionPolicy
25      from backend.pipeline.validators.base import ValidationResult
26
27      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
```

```


    def _pass():
        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]


    class _InlineExecutor:
        def submit(self, fn, *a, **k):
            fn(*a, **k)


    class _FakeCloudClient(CloudRunJobClient):
        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = (
                rows
                if rows is not None
                else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
            )

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump(
                        {
                            "video_id": self.video_id,
                            "segment_count": len(self.rows),
                            "status": "success",
                        },
                        f,
                    )
                with open(
                    os.path.join(outputs, "intervals.csv"),
                    "w",
                    newline="",
                    encoding="utf-8",
                ) as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(
                    {
                        "video_id": self.video_id,
                        "returncode": self.returncode,
                        "segment_count": len(self.rows),
                        "status": "success",
                    },
                    f,
                )
            return "exec-1"

```

```
        def cancel_execution(self, execution):
            raise AssertionError(
                "cancel must never fire on the happy path"
            )  # pragma: no cover


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
        db = Database(str(tmp_path / "t.db"))  # noqa: F841
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "db_instance", db)
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate(
            {  # noqa: F841
                "deployment": {"compute_target": "cloud_run"},
                "storage": {"backend": "local", "output_root": str(bucket)},
            }
        )
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(
            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
        )
        monkeypatch.setattr(
            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
        )
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
token. ``**kw``
        forwards the per-request ``target_override`` (item 3) so the cloud-picker path
resolves too."""
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(
                config, run_client=fake, policy=_FAST, token="tok", **kw
            ),
        )


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
```

```
150
151         out = m._execute_processing(
152             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
153         )
154
155         assert out["status"] == "success"
156         assert len(out["play_segments"]) == 2
157         rows = db.query_play_segments(out["video_id"], {})
158         assert len(rows) == 2
159         metas = [_meta(r) for r in rows]
160         assert all(md["source"] == "cloud_run" for md in metas)
161         assert {md.get("shot_count") for md in metas} == {5, 3}
162         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
163
164
165     def test_cloud_dispatch_builds_correct_spec(cloud_env):
166         _db, bucket, monkeypatch = cloud_env
167         fake = _FakeCloudClient()
168         _inject_fake(monkeypatch, fake)
169
170         m._execute_processing(
171             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172         )
173
174         assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
175         argv = fake.calls[0]
176         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177         assert argv[argv.index("--out-uri") + 1] == str(bucket)
178         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179         joined = " ".join(argv)
180         assert (
181             "deployment.compute_target=local_gpu" in joined
182         )  # container runs INLINE (never re-dispatch)
183         assert "indexing.detector_impl=native" in joined
184         assert (
185             "indexing.skip_ai_handoff=false" in joined
186         )  # the engine's AI-handoff stance is forwarded
187
188
189     def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
190         """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
it tries the
191         Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
is forwarded."""
192         _db, _bucket, monkeypatch = cloud_env
193         m.global_config.indexing.skip_ai_handoff = True
194         fake = _FakeCloudClient()
195         _inject_fake(monkeypatch, fake)
196         m._execute_processing(
197             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
198         )
199         assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])
200
201
202     def test_cloud_failure_marks_failed_no_rows(cloud_env):
203         db, _bucket, monkeypatch = cloud_env
204         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
205         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
206         fake = _FakeCloudClient(returncode=1)
207         _inject_fake(monkeypatch, fake)
208
209         out = m._execute_processing(
210             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
211         )
```

```
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []
        # It DID dispatch + run remotely (then failed rc=1), so the path is honestly
cloud_run + dispatched.
        assert (
            out["compute"]["path"] == "cloud_run" and out["compute"]["dispatched"] is True
        )


    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path


    def test_cloud_stages_local_upload_to_bucket(cloud_env, tmp_path, monkeypatch):
        """A browser upload lands on THIS box's local disk; under cloud-serving with a CLOUD
input_root it
        is STAGED to the bucket and the L4 job dispatches from the staged gs:// URI (not
hard-failed) ? #461.
        The DB video record is repointed at the staged source so a later /api/compile can
re-fetch it."""
        db, _bucket, _mp = cloud_env
        m.global_config.storage.input_backend = "gcs"
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri, kw.get("config")))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )

        assert out["status"] == "success"
        # 1) the local upload was staged UP to the input bucket, namespaced by video_id
        assert len(puts) == 1
        src, dest, put_cfg = puts[0]
        assert src == local_path
        assert dest.startswith("gs://in-bucket/videos/") and dest.endswith("/upload.mp4")
        # the active storage settings are threaded into put() (so a non-default S3 endpoint
/ GCS project /
        # etc. survives) ? not a fresh default client. The model_dump carries
input_root/input_backend.
        assert put_cfg is not None and put_cfg.get("input_root") == "gs://in-bucket/videos"
        assert put_cfg.get("input_backend") == "gcs"
        # 2) the L4 job dispatched from the STAGED gs:// URI, not the unreadable local path
        assert len(fake.calls) == 1
        argv = fake.calls[0]
```

```
271         assert argv[argv.index("--video-uri") + 1] == dest
272         # 3) the DB video record now points at the staged source (so compile re-fetches it)
273         row = db.get_video(out["video_id"])
274         assert row is not None and row["filepath"] == dest
275
276
277     def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
278         cloud_env, tmp_path, monkeypatch
279     ):
280         """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid
cloud config ?
```

11. user (2026-07-04T08:54:41.611Z)

```
Preparing worktree (detached HEAD d3a9629)
HEAD is now at d3a9629 review: thread active storage config into the staging put (#461)
```

12. user (2026-07-04T08:54:51.361Z)

```
 M tests/test_api_cloud_dispatch.py
```

13. user (2026-07-04T08:55:18.031Z)

```
Exit code 1
......F..................                                               [100%]
================================== FAILURES ===================================
____ test_cloud_stages_when_input_root_is_cloud_and_input_backend_default _____

cloud_env = (<backend.infrastructure.database.Database object at 0x00000200BC9178A0>, WindowsPat
h('C:/Users/avidu/AppData/Local/Te...u/pytest-1231/test_cloud_stages_when_input_r0/bucket'),
<_pytest.monkeypatch.MonkeyPatch object at 0x00000200BC962450>)
tmp_path = WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-1231/test_cloud_stages_when_input_r0')
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x00000200BC962450>

    def test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
        cloud_env, tmp_path, monkeypatch
    ):
        """input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid cloud
config ?
        the storage layer classifies an explicit root by its scheme (resolve_input_uri joins
under
        input_root before ever consulting input_backend), so the staging gate must too (PR #467
review).
        Regression: the gate previously required input_backend != 'local' and hard-failed this
shape."""
        _db, _bucket, _mp = cloud_env
        m.global_config.storage.input_root = "gs://in-bucket/videos"
        # input_backend deliberately NOT set ? stays at the model default 'local'
        assert m.global_config.storage.input_backend == "local"
        puts: list = []

        def _fake_put(src, uri, **kw):
            puts.append((str(src), uri))
            return uri

        monkeypatch.setattr(m.storage, "put", _fake_put)
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "upload.mp4")

        out = m._execute_processing(
            m.ProcessRequest(video_path=local_path, segmenter="wasb_hybrid")
        )
```

```
>       assert out["status"] == "success"  # NOT the 'needs a cloud input' pre-dispatch failure
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       AssertionError: assert 'failed' == 'success'
E
E         - success
E         + failed

tests\test_api_cloud_dispatch.py:303: AssertionError
============================== warnings summary ===============================
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================== short test summary info ==========================
FAILED tests/test_api_cloud_dispatch.py::test_cloud_stages_when_input_root_is_cloud_and_input_ba
ckend_default
1 failed, 25 passed, 4 warnings in 18.58s
```

14. user (2026-07-04T08:55:27.812Z)

```
(PowerShell completed with no output)
```

15. user (2026-07-04T08:55:51.606Z)

```
.........................                                                [100%]
============================== warnings summary ===============================
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
26 passed, 4 warnings in 16.00s
```

16. assistant (2026-07-04T08:55:58.614Z)

```
With the fix all 26 tests pass. Now verifying the fix agent's full-suite, ruff, and mypy claims
independently.
```

17. user (2026-07-04T08:56:00.711Z)

```
Command running in background with ID: bhcqyiz7c. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\bhcqyiz7c.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

18. user (2026-07-04T08:56:09.214Z)

```
All checks passed!
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 147 source files
```

19. user (2026-07-04T08:56:19.495Z)

```
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\config\models.py:406:    # =>
``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with ``input_backend``
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\config\models.py:410:
input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\config\presets.py:41:# these
bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:29:| D2 | **Two registries:** `ComputeBackend`
(where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by
`input_backend`/`storage.backend`. | ADR-014 | Realizes PLATFORM_ARCHITECTURE's
`local\|hosted\|byo_worker`. |
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:85:- Selecting a profile applies a **coherent
preset bundle** of *existing* knobs that today drift independently: **INPUT** (new
`input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new
`compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy),
**STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:94:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:117:| M2 | `input_backend`/`input_root`; wire
main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 |
safe | ? | [#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:160:- 2026-06-21 ? **M0 APPROVED**; **M1
landed** ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:161:- 2026-06-21 ? **M2 landed**
([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)):
`input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\README.md:172:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md:70:| M2 input_backend | L2 storage
roundtrip (4 backends); L3 worker fetch with local=identity |
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\docs\DATA_IN_GCS.md:120:A bare/relative
path resolves under a configured `input_root`/`input_backend`
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\orchestration.py:254:        #
Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged URI below is
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\orchestration.py:256:        #
so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left at its
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\orchestration.py:257:        #
default, while a bare/local-looking root can't stage no matter what input_backend says
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\orchestration.py:258:        #
(resolve_input_uri only consults input_backend when input_root is EMPTY, and an empty root
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\capabilities.py:69:
```

```
if sc.get("backend") == name or sc.get("input_backend") == name:
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:241:
m.global_config.storage.input_backend = "gcs"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:265:
# etc. survives) ? not a fresh default client. The model_dump carries input_root/input_backend.
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:267:
assert put_cfg.get("input_backend") == "gcs"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\tests\test_api_cloud_dispatch.py:277:def
test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:280:
"""input_root='gs://?' with input_backend left at its DEFAULT ('local') is a valid cloud config
?
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:282:
input_root before ever consulting input_backend), so the staging gate must too (PR #467 review).
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:283:
Regression: the gate previously required input_backend != 'local' and hard-failed this shape."""
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:286:
# input_backend deliberately NOT set ? stays at the model default 'local'
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:287:
assert m.global_config.storage.input_backend == "local"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_api_cloud_dispatch.py:317:
_db, _bucket, _mp = cloud_env  # fixture leaves input_root unset (input_backend=local)
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:76:    raw: str,
*, input_backend: str = "local", input_root: str = ""
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:85:      4. A
bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` + ``"bucket/x"`` ?
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:89:    So the
default (``input_backend="local"``, ``input_root=""``) is an identity passthrough for every
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:103:
input_backend and input_backend != "local"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\ref.py:105:
return f"{input_backend}://{s}"
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\__init__.py:52:
config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported URI scheme ?
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\__init__.py:60:
input_backend=sc.get("input_backend", "local"),
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\backend\storage\__init__.py:81:    non-
local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the local
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_input_source.py:4:The hard
guarantee: the default config (input_backend='local', input_root='') is a byte-for-byte
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_input_source.py:51:        #
4. input_backend prefixes a bare bucket/key when no input_root
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_input_source.py:66:
assert resolve_input_uri(raw, input_backend=ib, input_root=ir) == expected
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_input_source.py:88:    cfg =
{"input_backend": "local", "input_root": "gs://bucket/videos"}
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_input_source.py:217:
acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-
staging\docs\archives\past_projects\AUDIT_REPORT_khelsutra-
guru_2026-07-02-COMPLETED-2026-07-02.md:207:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_storage_capabilities.py:75:
assert cap._bucket_configured({"input_backend": "gcs"}, "gcs") is True
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_storage_gdrive_api.py:175:
{"storage": {"backend": "gdrive-api", "input_backend": "gdrive-api"}}
C:\Users\avidu\Projects\khelsutra-guru\wt-upload-staging\tests\test_storage_gdrive_api.py:178:
assert cfg.storage.input_backend == "gdrive-api"
```

20. user (2026-07-04T08:56:36.212Z)

```
1       ......................ss..........
```

21. assistant (2026-07-04T08:56:53.968Z)

The background suite run will notify on completion ? waiting for it before issuing the verdict.

22. user (2026-07-04T08:56:54.801Z)

```
1       ......................ss.............................................. [  4%]
2       ................................
```

23. assistant (2026-07-04T08:57:01.403Z)

Suite is at ~5% ? I'll pick this up when the background run completes.

24. user (2026-07-04T08:57:01.419Z)

[structured-output-enforce] You MUST call the StructuredOutput tool to complete this request.
Call this tool now.

25. user (2026-07-04T08:57:09.654Z)

```
1       ......................ss.............................................. [  4%]
2       ........................................................
```

26. user (2026-07-04T09:05:22.307Z)

Command running in background with ID: b2dv7amtt. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\tasks\b2dv7amtt.output. You will be notified when it completes. To check interim output, use Read on that file path.

27. user (2026-07-04T09:05:29.302Z)

```
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gcs://bucket/vide
os/x.mp4]
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1629 passed, 10 skipped, 8 warnings in 134.91s (0:02:14)
```

28. user (2026-07-04T09:05:56.510Z)

Structured output provided successfully